Topology-Preserving Sketch-Based Generation of 3D Assets

Selin Akmen Namozjon Ostonaev
Technical University of Munich
selin.akmen@tum.de namoz.ostonaev@tum.de

Abstract

Various studies have proposed different approaches to 3D object generation from hand-drawn sketches, including editing these objects by modifying the sketches, ranging from transformer-based vertex prediction to diffusion- and/or Neural Radiance Fields (NeRF)-based generation. However, existing approaches either lack diversity when the topology is well preserved or lose topology over iterations when diversity is high. To overcome these constraints, we leverage existing diffusion-based generation models to achieve high diversity and propose an attention-caching and masking mechanism for the latent to ensure topological stability. The evaluation results show that the geometry of the unedited region remains almost unchanged with our method.

1. Introduction

3D modeling has largely expanded beyond industrial engineering, ranging from asset iteration in game engineering to casual-hobbyist 3D printing. Current CAD applications are designed to produce complex, industry-oriented, mathematically representable 3D models, making it extremely hard to capture beyond industry, such as natural artifacts (e.g., non-flat surfaces). Generative models have huge potential, especially for 3D asset generation, to produce diverse, high-fidelity meshes with sophisticated textures that capture both human-made and natural artifacts. Existing approaches [4, 6] address this problem from different perspectives but still exhibit limitations in preserving topology [6] or lack diversity and texture [4]. These limitations motivate our research question: "Can we generate highly diverse and high-fidelity meshes through diffusion models, and edit parts while preserving the topology of the unedited part?" To address the question, we introduce our pipeline, which combines the image diffusion and image-to-3D generation models FLUX.2 [3] and TRELLIS [9] to produce textured meshes from sketches. We propose our KV-Cache Engine and Automatic Bounding Box (ABB) extraction method for localized topology-preserving editing based on the sketch edits. We evaluate our approach on the ShapeNet [1] dataset across a diverse set of object categories, using both quantitative and qualitative results to demonstrate its capabilities in mesh generation and topology-preserving editing.

2. Related Work

Multiple studies present different approaches to generating and editing 3D meshes.

The model SKED [6] enables sketch-guided text-based editing of 3D NeRF representations. It requires guiding sketches from 2 different views as an editing mask, along with an editing text prompt. The model estimates editing regions and generates edits using a pretrained diffusion model. It preserves unedited areas through loss constraints to keep density and color consistent and to localize editing only within the provided mask. However, the loss constraints primarily focus on appearance consistency rather than explicitly preserving mesh topology.

The more recent work MeshPad [4] approaches the editing task as an autoregressive mesh generation using a transformer-based architecture. The work decomposes editing into 2 tasks, addition and deletion, applied to mesh tokens. The model receives the color-coded sketch and predicts the mesh vertices corresponding to the edited region. Despite its strong performance in topology-preserving editing, its predictive capacity for 768 mesh faces limits the diversity.

VoxHammer [5] proposes a training-free approach to topology-preserving local editing in the 3D latent diffusion space. It predicts an inversion trajectory for a given 3D asset and stores latents and key-value pairs at each timestep to reconstruct the unedited area and generate the edited area, given as a text prompt, which provided the basis for the development of our proposed method.

3. Methodology

Our model consists of two pipelines: a standard generation pipeline to generate meshes from sketches, and an editing pipeline to update an existing mesh based on an edited sketch. To generate a mesh from a sketch, we lever-

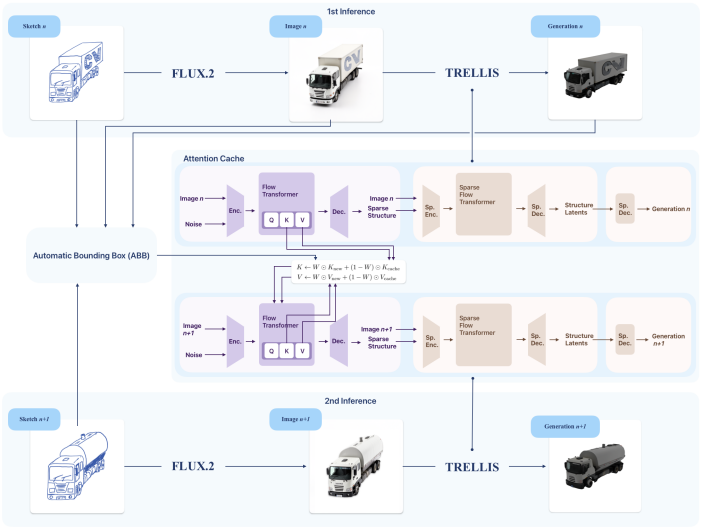

Figure 1. The visualization of our pipeline and KV-Cache Engine.

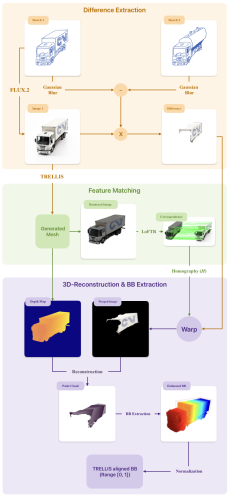

Figure 2. The visualization of our ABB extraction method.

age the pre-trained models FLUX.2 [3] and TRELLIS [9]. Since the sketches contain only structural information, we first generate a textured image using FLUX.2 while preserving the sketch geometry. From this image, we generate the mesh using TRELLIS. During mesh generation, we use our KV-Cache Engine, adapted from VoxHammer [5], to store the transformer key-value pairs required for subsequent editing. In the editing pipeline, we use previously generated assets, along with the previous and edited sketch, to extract the ABB, used as a mask for topology-preserving mesh editing. In the remainder of this section, we refer to the assets generated or used by the standard generation pipeline as "initial".

3.1. KV-Cache Engine

Our KV-Cache Engine (Fig. 1) provides a training-free and inversion-free method that strictly preserves the unedited regions of the mesh geometry. KV-caching is applied to the Flow Transformer component of TRELLIS [9], which is the DiT [7] responsible for structure generation, while it is not applied to the Sparse Flow Transformer component, which is the DiT responsible for texture generation. During the initial forward run, we cache the keys and values in the attention layer of the Flow Transformer. Using the spatial bounding box defined by the ABB, we generate a latent mask W. We then blend the newly generated keys and values with the cached pairs using the following formulation:

$$K \leftarrow W \odot K_{\text{new}} + (1 - W) \odot K_{\text{cache}} \quad (1)$$

$$V \leftarrow W \odot V_{\text{new}} + (1 - W) \odot V_{\text{cache}} \quad (2)$$

This operation dynamically updates the keys and values by replacing the unedited latent features with their cached counterparts, ensuring faithful reconstruction of the original mesh geometry in those regions.

3.2. ABB

To mark the edited area on the initial mesh, we propose a three-step ABB extraction method (Fig. 2). First, the difference between the initial and edited sketches is computed via pixel-wise subtraction after Gaussian smoothing, applied to improve robustness to small pixel-level variations. The resulting difference mask is applied to extract the textured difference from the initial image. Second, the initial mesh is rendered from multiple viewpoints, and the transformer-based feature-matching model LoFTR [8] is used to establish correspondences between each rendered view and the initial image. A coarse-to-fine view search method selects the best viewpoint based on the confidence values and the number of reliable matches and provides the homography matrix H. Third, using H and the depth map extracted during rendering, the 2D textured difference is projected into 3D space, creating a point cloud of the edited region. The point cloud is transformed into the mesh coordinate system, from which an axis-aligned bounding box is computed and normalized to the TRELLIS coordinate system for editing.

4. Results

4.1. Dataset

Fine-tuning. To fine-tune TRELLIS [9] for sketch-conditioned 3D mesh generation, we construct a dataset specifically tailored for training the Flow Transformer. We utilize the Objaverse dataset [2], selecting 100 objects from the chair category, and rendering 5 views per object.

Table 1. Evaluation of sketch-based mesh generation. The results of MeshPad evaluated on 500 meshes from 52 ShapeNet categories [1] are taken from [4, p.7]. The unit of CD is 0.001.

Method	CD ↓	CLIP ↑	FID ↓
MeshPad	**6.20**	**95.85**	**9.38**
Ours	36.33	87.88	52.01
Ours w/o noise	18.66	88.50	51.01

2D input sketches are extracted from rendered images using Canny edge detection. Following the standard TRELLIS pipeline, we extract the corresponding 3D ss_{latent} representations from the meshes, forming a total of 500 sketch-ss_{latent} pairs. Finally, the dataset is split into a 60/20/20 ratio for training, validation, and testing, respectively.

Evaluation. To evaluate our model's mesh generation performance, we use the ShapeNet [1] dataset by obtaining approximately 10 meshes per of the 52 categories. We render the meshes from five views: top-left, top-front-left, top-front, top-front-right, and top-right, and use them to create synthetic sketches via edge detection, following [4]. The sketches with excessive missing edge-detection and some frontal-view sketches are discarded due to inaccurate representations of the target geometry and insufficient visual information. After filtering, a single view is randomly selected for each mesh for evaluation.

We further assess our model's topology-preserving editing performance using nine manually edited sketches selected from our ShapeNet-derived sketch dataset, along with two additional hand-drawn edited sketches.

4.2. Fine-tuning Experiments

To adapt the model for sketch-to-structure generation, we fine-tune the Flow Transformer using Low-Rank Adaptation (LoRA). The LoRA configuration utilizes a rank of 32, an alpha of 16, and a dropout rate of 0.05. During training, we retain the Conditional Flow Matching (CFM) forward pass established in the original TRELLIS architecture. Specifically, we sample noise $\epsilon \sim \mathcal{N}(0, 1)$, predict the velocity field v, and use a Mean Squared Error (MSE) as a loss function. As a novel modification to enhance training stability, we apply a time-weighted penalty to the loss function. For any given timestep $t \in [0, 1]$, the loss is scaled by an exponential weighting factor of e^{-2t}.

4.3. Baseline and Metrics

Mesh Generation. We evaluate our model's mesh-generation performance by comparing it with MeshPad [4]. Since TRELLIS [9] occasionally generates noisy background geometry, such as floors, which adversely affect the alignment with the ground-truth meshes and cause an inaccurate assessment. Therefore, we perform the evaluation on both the complete synthetic dataset (Sec. 4.1) and a filtered version of the dataset without the noisy samples.

We assess the accuracy of the generated 3D shape geometry, the visual quality of rendered 3D outputs, and semantic alignment between the sketches and the generated meshes by computing the Chamfer Distance (CD), the Fréchet Inception Distance (FID), and CLIP similarity, respectively.

Table 2. Evaluation of geometry preservation during editing. The unit of masked CD is 0.001.

Method	Masked CD ↓
Ours	**0.15**
Baseline	3.16

Topology-Preserving Editing. We further assess our model's topology-preserving editing capability. As a baseline, we use the standard generation process described in Sec. 3. We compare the edited meshes generated with KV caching against the meshes generated without caching, using the same input sketch.

For evaluation, we use masked CD, computed only on the unedited regions of the generated meshes, to assess the geometric preservation of unedited regions during editing.

4.4. Quantitative Results

We evaluate our sketch-based mesh generation performance against MeshPad [4]. MeshPad achieves better CD, FID, and CLIP compared to our model (Tab. 1). Although evaluating on the filtered, noise-free dataset (Sec. 4.1) improves the CD score from 36.33 to 18.66 with little change in FID and CLIP scores, MeshPad still outperforms across all metrics.

We further evaluate our model's topology-preserving editing capability by measuring the geometry preservation of unedited regions of the sketches. As shown in Tab. 2, our method achieves a masked CD of 0.15, compared to 3.16 for the baseline, corresponding to approximately a 21-fold reduction in geometric error in the unedited regions.

4.5. Qualitative Results

Fig. 3 shows that our model can generate meshes closely resembling the ground-truth meshes and edit different parts of the sketches while preserving the unedited regions. However, it occasionally produces incomplete edits by modifying only part of the intended area, as illustrated by the lamp example (Fig. 4), and inaccurate edits within the correctly identified editing region, as exemplified by the chair sketch.

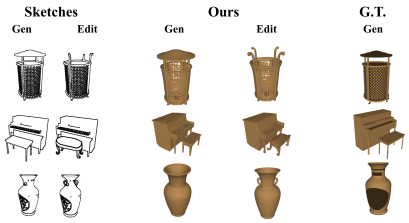

Figure 3. Visual results of our model for mesh-generation and editing tasks.

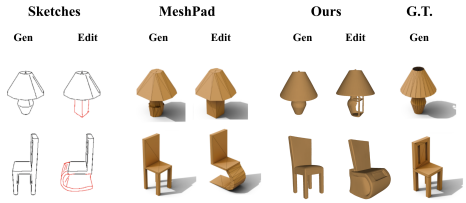

Figure 4. Visual comparison of our model with MeshPad [4]. The sketches, generated and edited meshes by MeshPad, and the ground truth meshes are taken from [4, p.6].

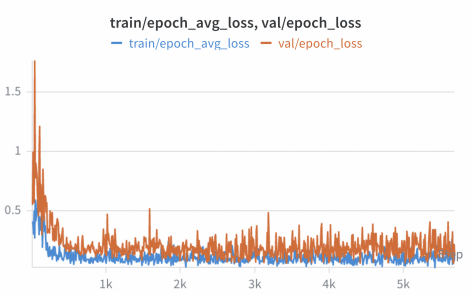

Figure 5. Visual graph of train and validation losses of fine-tuning.

5. Discussion

Our quantitative results for the topology-preserving editing task shown in Tab. 2 demonstrate that the proposed editing pipeline effectively preserves the geometry of the unedited regions. The masked CD of (0.15×10^{-3}) indicates that the unedited-region geometry remains nearly unchanged after editing. Compared with the baseline, our approach achieves approximately a 21-fold reduction in masked CD. The qualitative results for the topology-preserving editing task (Figs. 3 and 4) further illustrate that our method is capable of removing and updating the object parts.

By leveraging pretrained generative models FLUX.2 [3] and TRELLIS [9] for mesh generation, our approach can generate meshes with diverse shapes and appearances.

5.1. Limitations

Fine-Tuning Limitations. Our fine-tuning experiments began by trying to overfit a small subset of our dataset. Although the loss curve appeared promising (Fig. 5), the model failed to successfully overfit the target, oscillating around local minima and progressively degrading at later epochs (e.g., after epoch 500). We experimented with various interventions: tweaking the learning rate, LoRA hyperparameters, and weighted loss parameters; applying a learning-rate scheduler; shifting the noise distribution to be more 0.5-centric; and increasing the sample size to 100. Unfortunately, all trials revealed similar failure cases. We also verified our ground-truth ss_{latent} data by inverting to voxels and decoding to meshes. We hypothesize that the primary cause is hardware constraints that restricted our batch size to 2, which even gradient accumulation could not overcome.

Sketch-to-Image Limitations. Since we utilize the image-to-image generative model FLUX.2, there are certain limitations on how the model interprets sketches as solid images. In Fig. 4, second row, the edited result was caused by FLUX.2, since it generated the chair image with a solid bottom half.

ABB Limitations. Our ABB method has two main limitations. First, because it computes the bounding box (BB) from a 3D reconstruction of the changed part, it is inherently unable to capture regions behind rounded, spherical parts (Fig. 4, 1st row, our edited example). Second, the ABB mechanism utilizes the depth map from the initially generated mesh; thereby, it lacks depth information for the added part in the new sketch, making it unable to capture the addition's BB.

5.2. Future Work

Future work could extend our approach in several directions. First, texture preservation during editing could be enabled by leveraging the Sparse Flow Transformer in TRELLIS [9]. Second, to overcome the limitations of the pretrained models [9] and [3] in interpreting sketches, the models could be fine-tuned to better interpret the sketch inputs. Finally, the current ABB extraction method could be improved to better handle circular geometries and support a wider range of editing operations, including additions.

6. Conclusion

In this report, we presented a topology-preserving, sketch-based pipeline for 3D asset generation and editing that leverages diffusion-based image and 3D generation models. Attention-caching and masking mechanisms are introduced to preserve the unedited region's geometry. Although challenges remain in sketch interpretation performance, addition as an editing operation, and capturing spherical edited regions in meshes, our results show that our method produces diverse shapes and textures and effectively maintains the unedited geometry topology,

References

[1] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, et al. ShapeNet: An Information-Rich 3D Model Repository, 2015. 1, 3

[2] Matt Deitke, Dustin Schwenk, Jordi Salvador, Luca Weihs, Oscar Michel, Eli VanderBilt, Ludwig Schmidt, Kiana Ehsani, Aniruddha Kembhavi, and Ali Farhadi. Objaverse: A universe of annotated 3d objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13142–13153, 2023. 2

[3] Black Forest Labs. FLUX.2: Frontier Visual Intelligence. https://bfl.ai/blog/flux-2, 2025. 1, 2, 4

[4] Haoxuan Li, Ziya Erkoc, Lei Li, Daniele Sirigatti, Vladyslav Rozov, Angela Dai, and Matthias Nießner. Meshpad: Interactive sketch-conditioned artist-reminiscent mesh generation and editing, 2025. 1, 3, 4

[5] Lin Li, Zehuan Huang, Haoran Feng, Gengxiong Zhuang, Rui Chen, Chunchao Guo, and Lu Sheng. Voxhammer: Training-free precise and coherent 3d editing in native 3d space. *arXiv preprint arXiv:2508.19247*, 2025. 1, 2

[6] Aryan Mikaeili, Or Perel, Mehdi Safaee, Daniel Cohen-Or, and Ali Mahdavi-Amiri. Sked: Sketch-guided text-based 3d editing, 2023. 1

[7] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4195–4205, 2023. 2

[8] Jiaming Sun, Zehong Shen, Yuang Wang, Hujun Bao, and Xiaowei Zhou. Loftr: Detector-free local feature matching with transformers, 2021. 2

[9] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3d latents for scalable and versatile 3d generation, 2025. 1, 2, 3, 4